Functional and Technical Specification (SRS)

Algérie Télécom - 7.77 Platform

Online Training Session Management System - V1

Document type: Software Requirements Specification / Implementation Blueprint
Audience: Project manager, existing 7.77 platform developer, backend developer, frontend developer

Contents

1 Executive Summary 4
2 Project Context 5
2.1 Background 5
2.2 Current Manual Process 5
3 Existing User Data and Migration (Overview) 6
3.1 Password Migration - Security Analysis 6
4 New User Registration 7
4.1 Required Fields 7
4.2 Source vs. Derived vs. Editable Data 7
5 User Roles 8
5.1 Participant 8
5.2 Session Manager / Administrator - Should These Be Separate Roles? 8
6 Training Session Model - Core Fields 9
6.1 Multiple Target Age Categories 9
7 Training Identification: Training Type vs. Scheduled Session 10
7.1 The Problem 10
7.2 Recommended Model 10
7.3 Important Clarification for V1 10
8 Session Visibility 11
8.1 Eligibility Matching 11
8.2 Session List and Ordering 11
8.3 “Upcoming Sessions” vs. “My Sessions” 11
9 Session Status Lifecycle 12
9.1 Core States 12
9.2 Should FULL Be a Stored Status or Derived? 12
9.3 Time-Driven State Computation 12
10 Registration 13
10.1 Registration Checks and Concurrency Safety 13
11 Unregistration 14
11.1 Rule 14
11.2 Rationale and State Transition 14
12 Zoom Integration 15
12.1 Workflow 15
12.2 Security Considerations 15
13 Attendance 16
13.1 Definition (V1) 16
13.2 Idempotency 16
13.3 Separate Entity vs. Fields on Registration 16
14 Certificates 17
14.1 Workflow 17
14.2 Minimum Content 17
14.3 Client-Side vs. Server-Side Generation 17
14.4 Certificate Record 18
15 Email System 19
15.1 Required Capabilities for V1 19
15.2 Integration Approach 19
15.3 Configuration, Security, and Failure Handling 19
15.4 Background Job Requirement 19
16 Cancellation 20
16.1 Should CANCELLED Be Added to the Lifecycle? 20
16.2 Effects of Cancellation 20
17 Admin / Session Manager Dashboard 21

18 Scale 22
19 Technology Stack Evaluation 23
20 Security 25
20.1 Law 18-07 (Algerian Personal Data Protection Law) 26
21 Non-Functional Requirements 27
22 Database Design 28
22.1 Conceptual Model 28
22.2 ER Diagram 28
22.3 Key Constraints and Indexes 29
23 API Design (Representative Endpoints) 30
23.1 Authentication 30
23.2 Profile 30
23.3 Sessions 30
23.4 Registration 31
23.5 Attendance / Join 31
23.6 Certificates 31
23.7 Session Manager 31
24 Business Rules 32
24.1 Identified Ambiguities and Resolutions 32
25 Use Cases (Representative Selection) 33
25.1 UC-01: New Participant Registration 33
25.2 UC-06: Registering for a Session 33
26 User Journeys 34
27 State Diagrams 35
27.1 Session Lifecycle 35
27.2 Participant–Session Relationship 35
28 System Architecture 36
29 Background Tasks and Automation 37
30 Error Handling 38
31 Testing Strategy 39
32 Deployment 40
33 Data Migration 41
34 V1 Scope Exclusions and Rationale 42
35 Future V2 Direction (Non-Binding) 43
36 Implementation Roadmap 44
37 Acceptance Criteria (Representative) 46
38 Requirements Traceability Matrix (Representative Sample) 47
39 Key Design Decisions - Summary 49
40 Appendix A - Key Assumptions 51
41 Appendix B - Open Questions / TBD Items (Consolidated) 52
42 Appendix C - Recommended Technology Stack (Summary) 53
43 Appendix D - V1 / Future V2 Scope Recap 54

1 Executive Summary

Algérie Télécom's 7.77 platform currently manages online training sessions for participants aged 7–77 through a largely manual workflow: identifying eligible participants, contacting them, distributing Zoom links, checking attendance from recordings, and issuing certificates by hand. This document specifies a **new, independent web application** whose sole purpose in V1 is to **automate this existing workflow** - not to become a full Learning Management System (LMS).

The proposed system is a **modular monolith** built with **FastAPI (Python) + PostgreSQL** on the backend and **React** on the frontend, deployed on an organization-managed server. It introduces a clean separation between **Training Type** (the reusable identity of a training, e.g. "Introduction to AI") and **Session** (a single scheduled occurrence of that training), which allows the platform to recognize repeat attendance without restricting participant choice.

The system automates:

- Age-category-based session visibility and eligibility;
- Self-service registration/unregistration with a one-hour cutoff before session start;
- Concurrency-safe capacity enforcement;
- In-platform, access-gated Zoom link delivery (never emailed);
- Automatic attendance recording on first "Join Session" click;
- Certificate availability after administrator validation of attendance, with client-side generation recommended for V1;
- Reminder and cancellation notification emails, sent asynchronously.

A **one-time migration** will import existing participant accounts (23,000+) from the legacy platform. The exact migration mechanism is **TBD**, pending discussion with the legacy platform's team; this document defines the target data shape and a migration process, not the mechanism itself.

This document also documents **20 explicit design decisions** (Section 38) with options, trade-offs, and recommendations, since many of these choices materially affect the database schema and API design and should be signed off before implementation begins.

---

2 Project Context

2.1 Background

The 7.77 platform serves participants across seven age categories:

Category	Age range
C1	7–10
C2	11–14
C3	15–17
C4	18–23
C5	24–30
C6	31–45
C7	46–60

2.2 Current Manual Process

1. Administrator prepares/announces a session.
2. Eligible participants are identified manually by age category.
3. Contact information is retrieved manually.
4. Participants are contacted by email/phone.
5. Registration is collected manually.
6. The Zoom link is emailed manually.
7. The session takes place.
8. Attendance is inferred manually from the session recording.
9. Certificates are prepared manually.
10. Certificates are emailed manually.
11. Previously-attended participants are manually deprioritized.

---

3 Existing User Data and Migration (Overview)

The legacy platform stores, at minimum: user ID, name, email, phone, age category, password. The new platform will:

- Perform a **one-time** import of this data;
- Operate **fully independently** afterward (no live dependency on the legacy platform, no shared code/services);
- Use **email as the natural unique identifier** going forward, while preserving the legacy `user_id` as a reference field (`legacy_user_id`) for traceability and support/debugging during the migration window.

The mechanism for extracting legacy data (DB export, API, CSV/JSON) is **TBD** and depends entirely on what the legacy team can provide - see Section 32 for the process this document assumes regardless of mechanism.

3.1 Password Migration - Security Analysis

Three scenarios are possible, depending on how the legacy platform stores passwords:

Scenario	Description	Migration feasibility
A	Legacy passwords are hashed with a modern algorithm (bcrypt, Argon2, scrypt) with per-user salt	**Migratable**: copy the hash directly; the new system verifies against it on first login, then optionally re-hashes with its own configured algorithm.
B	Legacy passwords are hashed with a weak/legacy algorithm (MD5, SHA1, unsalted SHA256)	**Not recommended to migrate as-is.** The hash itself can be imported, but must be treated as **insecure** and force a reset.
C	Legacy passwords are stored in plaintext or reversibly encrypted	**Must not be migrated.** Force account activation via reset link regardless of effort saved.

Recommendation: Do not assume Scenario A. Request confirmation of the legacy hashing algorithm from the platform team as a prerequisite migration question (see Section 32). Regardless of the answer, implement a **secure account activation flow** (email a time-limited, single-use activation link that lets the migrated user set a new password) as the default and safest path for **all** migrated accounts. This avoids depending on unverifiable legacy security practices and gives every migrated user a fresh credential in the new system. If Scenario A is confirmed and stakeholders prefer not to force a reset for UX reasons, hash migration may be used as a fallback, but the activation-link approach is the recommended default.

---

4 New User Registration

4.1 Required Fields

Name, email, phone number, date of birth, age category (derived), wilaya, password.

4.2 Source vs. Derived vs. Editable Data

Field	Classification	Notes
Name	Source (user-entered)	Editable
Email	Source (user-entered)	Editable; must remain unique; used as login identifier
Phone number	Source (user-entered)	Editable
Date of birth	Source (user-entered)	Editable, but changes recompute age category (see 3.3)
Age category	**Derived** from date of birth	Not directly editable by the user in the recommended design (see decision below)
Wilaya	Source (user-entered)	Editable
Password	Source (user-entered, hashed at rest)	Editable via change-password flow
`legacy_user_id`	Source (migration only)	Not editable, not shown as an active field to new registrants

5 User Roles

5.1 Participant

register, log in, manage account, view sessions, register/unregister, access Zoom through the platform, have attendance recorded, view history, obtain certificates.

5.2 Session Manager / Administrator - Should These Be Separate Roles?

create/publish/modify/cancel sessions, configure category/capacity/Zoom URL, monitor registrations, view aggregates.

Recommendation: Use a **single role,** `SESSION_MANAGER`, for V1. Introducing a separate `ADMIN` role now would require speculative decisions about what an admin can do that a session manager cannot (user management? category configuration? nothing in the requirements justifies this split yet). The role model should, however, be implemented as an extensible enum/table (not hardcoded booleans) so that a genuine `ADMIN` role can be added later (e.g., for managing the `AgeCategory` reference table, or for future user administration) without a schema migration that breaks existing authorization checks.

A user with role `SESSION_MANAGER` can also be a participant in principle (e.g., an instructor who also registers for other sessions), so the role should be modeled as an attribute of the user (or a many-to-many `user_roles` table), not a mutually exclusive account type.

---

6 Training Session Model - Core Fields

Field	Type	Notes
`session_id`	UUID / surrogate PK	Unique scheduled-occurrence identifier
`training_id` (FK)	reference	Links to reusable Training Type
`title`	string	Display title; NOT used as identity
`description`	text	
`date`	date	
`start_time`	time	
`duration_minutes`	integer	
`target_age_category`	FK / FK-list	See 5.1
`max_capacity`	integer, default 100	
`zoom_url`	string, access-restricted	See Section 11
`instructor_name`	string	
`status`	derived/computed, see Section 8	
`created_at` / `updated_at`	timestamp	Audit fields

6.1 Multiple Target Age Categories

many-to-many session↔category (generalized model): Slightly more schema/query complexity (a join table `session_target_categories`) but allows a future session to be opened to two adjacent categories without a schema change, and costs nothing extra when only one category is used per session (the join table simply has one row).

---

7 Training Identification: Training Type vs. Scheduled Session

This is one of the most important modeling decisions in the system.

7.1 The Problem

A session **title** is free text and cannot reliably answer "has this participant already attended **this** training before?" because titles can be typo'd, reworded, or reused inconsistently by different session managers.

7.2 Recommended Model

Introduce two distinct entities:

- **`TrainingType`** - the reusable identity of a training program (e.g., `AI-INTRO-001`, name "Introduction to Artificial Intelligence", optional description/category tag). Created once, reused across many scheduled occurrences.
- **`Session`** - one scheduled occurrence of a `TrainingType` (a specific date, time, instructor, Zoom URL, capacity). Every `Session` has a mandatory FK to exactly one `TrainingType`.

`Registration` and `Attendance` are then naturally tied to `Session`, and by extension to `TrainingType` through the FK - allowing the query "has participant X attended any Session under TrainingType AI-INTRO-001?" to be a simple join, without needing a separate denormalized table.

This distinction should be visible to session managers when creating a session (they pick an existing `TrainingType` or create a new one) and used in the participant UI as a small badge: "You have already attended this training" next to a session whose `TrainingType` the participant has previously attended and completed.

7.3 Important Clarification for V1

Per Section 6 of the requirements, this identification is **for information/UX purposes only** in V1 - it must **not** block or deprioritize registration. The participant remains free to register for any eligible session regardless of prior attendance of the same `TrainingType`. This is a deliberate reversal of the old manual process's prioritization behavior, and should be called out clearly in the business rules (Section 23) so it is not accidentally "fixed" into a restriction during implementation.

---

8 Session Visibility

8.1 Eligibility Matching

A session is visible to a participant if the participant's `age_category_id` matches one of the session's `target_age_category` entries (Section 5.1).

8.2 Session List and Ordering

Participants see **all sessions relevant to their category**, regardless of registration availability, ordered:

`OPEN → RUNNING → FULL → COMPLETED`

Within each group, secondary ordering by date/time (soonest first) is recommended, **recommendation**: sort ascending by `date, start_time` within each status group, since participants most likely want to see the soonest opportunity first within "OPEN," and the most recently finished within "COMPLETED" (descending) for relevance.

8.3 "Upcoming Sessions" vs. "My Sessions"

- **Upcoming Sessions:** all sessions eligible for the participant's category, in the above order, **excluding** COMPLETED sessions the participant did not register for (to avoid clutter) - but including OPEN/RUNNING/FULL sessions they haven't registered for, so they can still register or observe capacity.
- **My Sessions:** only sessions the participant has a `Registration` record for, with the following distinct display states derived from the participant-session relationship (not the session's own lifecycle status):

Display state	Meaning	Derived from
Registered	Registration exists; session not yet started	`Registration.exists AND Session.status IN (OPEN, FULL)`
Ongoing	Session currently running; participant registered	`Registration.exists AND Session.status = RUNNING`
Attended	Session completed; attendance recorded	`Registration.exists AND Attendance.exists AND Session.status = COMPLETED`
Absent	Session completed; no attendance recorded	`Registration.exists AND Attendance.does_not_exist AND Session.status = COMPLETED`
Cancelled	Session was cancelled after registration	`Session.status = CANCELLED`

This correctly separates **session completion** (a property of the session) from **attendance** (a property of the participant-session relationship), as required.

---

9 Session Status Lifecycle

9.1 Core States

`OPEN → RUNNING → COMPLETED`, plus `CANCELLED` as a terminal override (Section 15), plus the derived UI condition `FULL`.

9.2 Should FULL Be a Stored Status or Derived?

Recommendation: Derived, not stored.

Reasoning:

- `FULL` is not a point in time the way OPEN/RUNNING/COMPLETED are; it is a **capacity condition** that can become true and then false again (if a registration is later cancelled, freeing a spot, the session should immediately become registrable again).
- Storing it as a fourth lifecycle value would require an extra write on every registration/unregistration to keep it in sync, introducing a race condition risk on top of the concurrency control already required for capacity (Section 9).
- Deriving it (`registered_count >= max_capacity` computed at read time, or maintained via a fast counter kept consistent by the same transaction that enforces capacity) keeps a single source of truth.

Implementation detail: maintain a `registered_count` integer column on `Session`, updated transactionally within the same DB transaction that inserts/deletes a `Registration` row (see Section 9.1), and compute `is_full = registered_count >= max_capacity` at query/serialization time. This avoids a `COUNT()` on every list request while still keeping FULL a derived, always-consistent value rather than an independently-writable status.

9.3 Time-Driven State Computation

Time relative to session	Status	Zoom access
Before `start_time - 60 min`	OPEN (or FULL if capacity reached)	Not available
`start_time - 60 min` to `start_time`	OPEN (or FULL)	**Available**
`start_time` to `start_time + duration`	RUNNING	Available
After `start_time + duration`	COMPLETED	Not available
Any time, if cancelled	CANCELLED	Never available

Recommendation: compute `status` **on read** (in the API layer / a SQL view or a computed property), driven purely by `now()` vs. `date + start_time + duration`, rather than via a scheduled job that writes a `status` column. This guarantees the status is always correct even if a background job is delayed or fails, at the cost of a small amount of computation per request (negligible at this scale - see Section 20). `CANCELLED` remains the one status that **is** stored, since it is an explicit administrative action, not a function of time (Section 15).

---

10 Registration

10.1 Registration Checks and Concurrency Safety

On "Attend," the backend verifies, in order, inside a single database transaction:

1. User is authenticated (JWT/session valid).
2. Session exists and is not `CANCELLED`.
3. Participant's `age_category_id` is among the session's target categories.
4. Session is not `COMPLETED` (computed from time).
5. No existing `Registration` for (user_id, session_id).
6. Capacity not reached.

Concurrency-safe capacity enforcement - recommended implementation:

```
BEGIN;
SELECT max_capacity, registered_count
  FROM sessions
  WHERE id = :session_id
  FOR UPDATE;              -- row lock prevents concurrent over-booking

-- application checks registered_count < max_capacity
INSERT INTO registrations (user_id, session_id, registered_at) VALUES (...);
UPDATE sessions SET registered_count = registered_count + 1 WHERE id = :session_id;
COMMIT;
```

A `SELECT ... FOR UPDATE` row lock on the session row, combined with a `UNIQUE(user_id, session_id)` constraint on `registrations` (to guarantee rule 5 even under a race), is sufficient at this scale (hundreds of concurrent requests per session, not thousands per second) and avoids introducing distributed-locking infrastructure. An additional **database-level CHECK-style safeguard** - a `CHECK (registered_count <= max_capacity)` constraint, or simply re-validating capacity inside the same locked transaction as shown above - ensures capacity is never exceeded even under bugs in application logic.

The frontend displays `registered_count / max_capacity` (e.g., `73/100`) and disables the "Attend" button with a `Registration Full` label once `registered_count >= max_capacity`, but this is a UX convenience only - the authoritative check is always server-side.

---

11 Unregistration

11.1 Rule

A participant may unregister any time **before** `start_time - 60 minutes`. Once the Zoom-access window opens (i.e., from `start_time - 60 min` onward), unregistration is disabled.

11.2 Rationale and State Transition

This cutoff exists because the Zoom access window and the unregistration cutoff are **intentionally the same instant** - once a participant could potentially have already accessed the meeting, allowing "unregistration" afterward would create an inconsistent state (attended-but-unregistered). Tying both to one shared time boundary (`start_time - 60 min`) keeps the rule simple to explain and simple to implement (a single computed boundary reused in two checks).

```
Registered --(unregister, only if now < start_time - 60min)--> No registration
Registered --(time passes start_time - 60min)--> Registered, unregistration locked, Zoom access opens
```

If a participant does not unregister and simply never joins, they become `Absent` once the session completes (Section 7.3) - no special "no-show" handling is required beyond this derived state.

---

12 Zoom Integration

12.1 Workflow

Session managers create the Zoom meeting externally and paste the URL into the session form. The platform never emails this URL; it is retrieved only through the "Join Session" action, which simultaneously records attendance (Section 12) before revealing/redirecting to the URL.

12.2 Security Considerations

Risk	Mitigation
Zoom URL leaked via page source / API response to ineligible users	The `zoom_url` field is **never included** in the general session list/detail API response. It is only returned by a dedicated `POST /sessions/{id}/join` endpoint, which re-runs the full eligibility+registration+time check server-side before returning it.
URL shared by a legitimate participant with an unregistered third party	Acceptable residual risk for V1 (same risk exists with any meeting link); mitigated somewhat by Zoom's own waiting-room/passcode features, which are outside this platform's control since meetings are created externally. Documented as a known limitation, not solved by the platform.
Replay / direct API calls bypassing the UI	The `/join` endpoint must independently re-validate authentication, registration, eligibility, and the time window server-side - the frontend "Join Session" button is not trusted as the source of authorization.
URL access before the 1-hour window	Endpoint returns a `403` /structured error with a "come back at HH:MM" message rather than the URL, until the window opens.

---

13 Attendance

13.1 Definition (V1)

Attendance = first successful call to the Join action while all eligibility/registration/time checks pass. Not based on recording review, Zoom reports, or duration.

13.2 Idempotency

First click: create attendance record, `attended_at = now()`. Subsequent clicks: return the existing attendance record / redirect to Zoom again without creating a duplicate row or changing `attended_at`. This is naturally enforced by a `UNIQUE(registration_id)` (or `UNIQUE(user_id, session_id)`) constraint on the attendance table combined with an "insert if not exists" (`INSERT ... ON CONFLICT DO NOTHING`) pattern.

13.3 Separate Entity vs. Fields on Registration

Option A - fields on `Registration`: add `attended_at`, `attendance_validated` columns directly to the registration row.

Option B - separate `Attendance` entity, one-to-one (or one-to-zero-or-one) with `Registration`.

Criterion	Option A	Option B
Simplicity for V1's exact scope	Simpler; fewer joins	Slightly more joins
Conceptual clarity (registration ≠ attendance, per explicit requirement)	Blurs the distinction at the schema level	Matches the required conceptual separation exactly
Extensibility (future: partial attendance duration, multiple check-ins, attendance-validation audit trail with validator user + timestamp)	Requires later schema migration/ column bloat	New columns/rows added to `Attendance` without touching `Registration`

Recommendation: Option B, a dedicated `Attendance` entity (PK `registration_id`, `attended_at`, `validated_at`, `validated_by` FK to the session manager who validated it - see Section 13). This directly mirrors the requirement to never conflate "registered" and "attended," costs one extra join, and leaves room for the certificate-validation workflow (Section 13) to attach its own fields without overloading `Registration`.

---

14 Certificates

14.1 Workflow

`Session COMPLETED → Attendance eligible for validation → Session Manager validates → Certificate becomes available for download`

14.2 Minimum Content

Participant name, session title, session date (all obtainable from existing `User` / `Session` data - no separate denormalized "certificate content" storage is required beyond a record that a certificate **has been** issued, described below).

14.3 Client-Side vs. Server-Side Generation

Criterion	Client-side (browser-generated PDF)	Server-side (backend-generated PDF)
Infrastructure cost	None - no PDF library/service on backend	Requires a PDF-generation library (e.g., WeasyPrint, ReportLab) and CPU time on the server
Consistency of visual output across browsers/devices	**Risk**: font rendering, canvas/PDF libraries (e.g., `pdf-lib`, `jsPDF`) can render slightly differently across browsers; template fidelity to an existing PDF/Word/Canva design may be harder to reproduce pixel-perfectly	High - server renders once from a controlled environment, guaranteeing identical output regardless of the participant's device
Reliability / auditability	Nothing is stored - if the participant loses the file, it must be regenerated identically from the same inputs (fine, since content is deterministic)	Same - but the backend can optionally persist the generated file or at least a "certificate issued" record with a checksum
Security	Template and generation logic exposed in browser bundle (not sensitive, since certificates aren't secret, but tamper risk: a participant could self-generate a certificate for a session they didn't attend, by manipulating client-side code, unless the certificate data itself is **fetched** from an authenticated, server-validated endpoint)	Backend validates attendance before generating; presented data cannot be tampered with client-side
Implementation effort for 2-person team	Lower short-term effort, but a real risk of "looks slightly different per browser" defeats the point of a certificate that should look identical for everyone	Moderate effort, well-understood pattern (existing template + a Python HTML-to-PDF library)

Recommendation: Server-side generation, for one decisive reason: a certificate is a document of record ("you can prove you attended"), and its authenticity depends on the backend being the party that both **validates attendance** and **produces the artifact**. Purely client-side generation means the "generation" step trusts client-supplied data for something that should be tamper-evident, and it also risks visual inconsistency with the organization's existing certificate template (PDF/Word/Canva).

A pragmatic middle ground, if the team wants to avoid PDF-rendering complexity: the backend exposes a **validated data endpoint** (`GET /certificates/{id}` returning JSON: name, training title, session date, validated=true) and the **frontend renders the PDF client-side from that trusted, server-validated payload** using a fixed, single, professionally-designed template (built once by the frontend developer to match the existing certificate design). This captures most of the "no server PDF library" simplicity while removing the tamper-and-consistency risk, since the data itself is authenticated even though the file rendering happens in the browser. **This hybrid approach is the specific V1 recommendation**, revisited only if the visual fidelity proves inadequate in testing, in which case fall back to full server-side rendering.

No admin template-upload system is justified for V1 - the template is fixed and built directly into the frontend/backend code as an asset, and changing it is a code change, not a runtime configuration feature (Section 13, explicit instruction against over-engineering this).

14.4 Certificate Record

Even with client-rendered PDFs, the backend should persist a minimal `Certificate` record (`id`, `registration_id` FK, `issued_at`, `validated_by`) so that "has this participant received a certificate for this session" is queryable and idempotent, and so a future "email me my certificate" feature (Section 14) has something concrete to attach to an email.

---

15 Email System

15.1 Required Capabilities for V1

Email type	Priority	Complexity
Session reminder (e.g., 24h and/or 1h before)	Must Have	Low - templated, triggered by scheduled check
Session cancellation notice	Must Have	Low
Account activation (migrated users)	Must Have	Low–Medium - needs secure token generation/expiry
Password reset	Must Have	Low–Medium - same token infrastructure as activation
Registration confirmation	Should Have	Low
Certificate email	Could Have	Low, once certificate data endpoint exists (Section 13.4)

15.2 Integration Approach

TBD: whether the organization provides SMTP infrastructure under `@algerietelecom.dz`, or whether the application must use an external transactional email provider. Since the current team already sends emails with Zoom links programmatically, **the recommended first step is to reuse whatever SMTP credentials/mechanism that existing code already uses**, rather than provisioning a new email service - this is the path of least new infrastructure. If that is unavailable to the new application for organizational reasons, a standard SMTP-compatible transactional provider is a reasonable fallback, to be selected once the organization's constraints are known.

15.3 Configuration, Security, and Failure Handling

- SMTP credentials stored as environment-managed secrets (Section 19), never in source control.
- Emails sent over TLS (SMTPS/STARTTLS).
- Activation/reset tokens: cryptographically random, single-use, time-limited (e.g., 24h for activation, 1h for password reset), stored hashed (not plaintext) in the database, matching password-reset security best practice (Section 19).
- **Failure handling:** email sending must be decoupled from the core transaction it relates to. E.g., a registration must succeed in the database even if the confirmation email fails to send. Failures should be logged and, for reminders/cancellations, retried with backoff up to a small limit; a failed email must never roll back or block the underlying business action (registration, cancellation, certificate validation).
- The Zoom URL is never included in any email body, per explicit requirement (Section 11).

15.4 Background Job Requirement

Emails are inherently asynchronous (should not block the HTTP request that triggers them) - see Section 28 for the recommended lightweight implementation (no Celery/Redis needed at this scale).

---

16 Cancellation

16.1 Should CANCELLED Be Added to the Lifecycle?

Yes. Unlike `FULL` (Section 8.2), `CANCELLED` is not a time-derived condition - it is the result of an explicit administrative action that must **override** whatever the time-derived status would otherwise be (e.g., a session could be cancelled while still technically "OPEN" by the clock). It must therefore be a **stored** field (`Session.cancelled_at`, nullable, or a boolean `is_cancelled` + `cancelled_at`), checked **before** the time-derived computation in Section 8.3: if `is_cancelled`, status is `CANCELLED` regardless of what the clock says.

16.2 Effects of Cancellation

- Session becomes non-joinable immediately (the `/join` endpoint checks `is_cancelled` first and rejects).
- All existing `Registration` rows for the session trigger a cancellation email (asynchronously, Section 14).
- The session remains visible in "My Sessions" / session lists with a `CANCELLED` badge (not deleted), for transparency and record-keeping.
- Unregistration and new registration are both disabled once cancelled.

---

17 Admin / Session Manager Dashboard

Scope exactly matches the requirement: session CRUD (create/edit/cancel), category/capacity/Zoom URL configuration, registration and attendance counts per session (e.g., **Registered: 86, Attended: 73, Absent: 13** - all derived as in Section 7.3), and attendance validation (Section 13). No search, filtering, export, or manual attendance correction in V1 - these are explicitly deferred (Section 33) because they are not required to eliminate the manual steps this project targets.

---

18 Scale

Metric	Value
Existing users to migrate	~23,000+
Sessions/month	~24 = 6 x 4 / month
Typical session capacity	100, up to 300
Peak concurrency pattern	Bursty, around each session's start time (registration bursts, and a "join" burst in the hour before start)

This is a modest scale (tens of thousands of users, dozens of sessions/month, low-hundreds of concurrent actions at peak) that a single well-indexed PostgreSQL instance and a single (or few, load-balanced) FastAPI process comfortably handle. See Section 27 for architecture sizing.

---

19 Technology Stack Evaluation

Layer	Choice	Assessment
Backend framework	**FastAPI**	Appropriate: async-capable, strong typing/validation via Pydantic, good fit for a Python-experienced solo backend developer, mature ecosystem.
Database	**PostgreSQL**	Appropriate: strong transactional guarantees (needed for capacity concurrency, Section 9), mature, well-supported by the target scale.
Frontend	**React**	Appropriate: standard choice, large ecosystem, matches frontend developer's stated preference.
ORM (Object-Relational Mapping)	SQLAlchemy 2.x (with Alembic for migrations)	Recommended: the de facto standard for FastAPI + PostgreSQL, mature transaction/locking support (needed for `SELECT ... FOR UPDATE`, Section 9.1).
Validation	Pydantic v2 (built into FastAPI)	Already implied by FastAPI; used for both request/response schemas and settings management.
Authentication	JWT access token (short-lived) + refresh token (httpOnly cookie)	See Section 38, decision 15, for full analysis.
Background jobs	In-process `asyncio` scheduled tasks / a lightweight scheduler library (e.g., APScheduler)	See Section 28 - no Celery/Redis needed at this scale.
Certificate rendering	Client-side, from a server-validated data endpoint (Section 13.3)	-
Frontend↔Backend communication	REST/JSON over HTTPS, `fetch` / `axios` from React	Matches the API design in Section 22.
Logging	Structured JSON logs (Python `logging` + a formatter), shipped to files or a lightweight log collector	See Section 19/20.
Configuration	Environment variables (`.env` for local dev, real environment variables/secret store in production), loaded via Pydantic `BaseSettings`	-
Deployment	Single organization-managed server; reverse proxy (Nginx) + systemd-managed FastAPI (via Uvicorn/Gunicorn workers)	See Section 31 for the container vs. non-container discussion.
Testing	pytest (backend), React Testing Library/Vitest or Jest (frontend), Playwright/Cypress (E2E)	See Section 30.

Monitoring	Application logs + basic uptime/health-check endpoint; a lightweight metrics tool if the organization already has one available	See Section 20.
Backups	PostgreSQL `pg_dump`/WAL-based backups on a schedule	See Section 31.

Overall assessment: The proposed stack is appropriate and is **not** over-specified for the problem. A **modular monolith** (a single FastAPI application organized into clear internal modules: `auth`, `users`, `sessions`, `registrations`, `attendance`, `certificates`, `notifications`) is recommended over microservices - there is no scale, team-size, or independent-deployment justification for splitting services, and doing so would multiply operational complexity for a two-person team with no dedicated DevOps function.

---

20 Security

Area	Requirement
Password hashing	Argon2id or bcrypt (via `passlib`), never reversible encryption.
Authentication	JWT-based, short-lived access tokens + refresh token rotation (Section 38.15).
Authorization	Role-based checks (`PARTICIPANT` / `SESSION_MANAGER`) enforced server-side on every protected endpoint, not just hidden in the UI.
Transport	HTTPS enforced everywhere (HSTS); no plaintext HTTP in production.
Input validation	Pydantic schemas for every request body; strict types, length limits, and format checks (email, phone, date).
SQL injection	Mitigated by using the ORM's parameterized queries exclusively; no raw string-interpolated SQL.
XSS	React's default escaping of rendered content; avoid `dangerouslySetInnerHTML`; sanitize any user-supplied text (e.g., session description) if HTML is ever allowed.
CSRF	Primarily relevant if using cookie-based auth; if JWT is sent via `Authorization` header (recommended, see 38.15) CSRF risk is greatly reduced. If a refresh token is stored in an httpOnly cookie, apply `SameSite=Strict/Lax` and a CSRF token for state-changing cookie-authenticated requests.
Session/token handling	Short-lived access tokens (e.g., 15 min), refresh tokens rotated and revocable, tokens never logged.
Rate limiting	Apply to login, registration, password-reset request, and join endpoints to blunt brute-force and enumeration attempts.
PII protection	Name/email/phone/DOB/wilaya treated as personal data; access restricted to the owning user and authorized session managers where strictly needed for their function (e.g., session managers should not need to see phone numbers unless a stated business need exists - flagged as **TBD**: confirm whether session managers require participant PII visibility beyond aggregate counts, given V1 explicitly excludes participant search/export).
Zoom URL protection	Never in list/detail responses or emails; served only via the validated `/join` endpoint (Section 11.2).
Certificate access	Only the owning participant (and the issuing session manager, for audit) can retrieve a given certificate's data; endpoint requires authentication and ownership check.
Email security	TLS to SMTP relay, credentials as secrets, activation/reset tokens hashed at rest and single-use.
Logging	No plaintext passwords, tokens, or full PII payloads in logs; log identifiers (user ID, session ID) rather than raw personal data where possible.
Secrets management	Environment variables / a secret manager if the organization provides one; never committed to source control.
Database security	Least-privilege DB user for the application (no superuser), network-restricted access, encrypted backups.
Account enumeration	Login and password-reset-request endpoints return generic responses ("if this account exists, an email has been sent") to avoid confirming which emails are registered.
Password reset security	Single-use, short-lived, hashed tokens; invalidate all other active sessions/tokens on password change.

20.1 Law 18-07 (Algerian Personal Data Protection Law)

This document does not provide legal advice. The following **technical** points should be reviewed with the organization's legal/privacy team against Law 18-07's requirements:

- Confirming the legal basis and retention period for storing participant personal data (name, email, phone, DOB, wilaya) and attendance history.
- Whether explicit consent language is required at registration and during migration (since migrated users did not register directly in the new system).
- Data subject rights (access, correction, deletion) and how the platform would technically support a deletion/export request if legally required.
- Whether personal data may be processed/stored outside Algeria if any third-party email or hosting service is used (relevant to the SMTP provider decision, Section 14.2).

These are flagged as **TBD**, to be resolved with legal/security stakeholders before production launch, not implementation blockers for V1 development itself.

---

21 Non-Functional Requirements

Category	Requirement
Performance	The system must remain responsive (sub-second typical API response) under normal load and specifically during the hour before a session, when a burst of "Join Session" requests is expected (up to 300 for the largest sessions).
Scalability	Comfortably support 23,000+ registered users and 100–300 participants/session on a single well-indexed PostgreSQL instance and a small number of FastAPI worker processes; architecture should allow adding worker processes/horizontal replicas behind the reverse proxy if needed later.
Availability	The platform should be available and stable around all scheduled session start times; scheduled maintenance should avoid known session windows.
Reliability	Email delivery failures and certificate-generation issues must never corrupt or block registration/attendance records (Section 14.3); these are decoupled, asynchronous, and independently retryable.
Maintainability	Modular monolith with clear internal module boundaries (Section 18), migrations managed via Alembic, consistent API error format (Section 29).
Usability	Participant interface limited to the flows actually needed (Upcoming Sessions, My Sessions, session detail/join, profile, certificates) - no unnecessary navigation depth.
Accessibility	Semantic HTML, sufficient color contrast, keyboard navigability for forms and buttons, alt text for any images (e.g., certificate preview), and adherence to basic WCAG 2.1 AA guidance for the participant-facing screens.
Observability	Structured logs for authentication events, registration/unregistration, attendance, certificate validation, and email send outcomes; a basic `/health` endpoint for uptime checks.
Data integrity	Registration/capacity operations are transaction-safe (Section 9.1); attendance is idempotent (Section 12.2); cancellation is atomic with respect to notification triggering.

---

22 Database Design

22.1 Conceptual Model

Core entities: `User`, `Role`, `AgeCategory`, `TrainingType`, `Session`, `SessionTargetCategory` (join table), `Registration`, `Attendance`, `Certificate`, `NotificationLog`, plus supporting `PasswordResetToken` / `ActivationToken`.

Are all of these necessary? Yes, with one simplification: `Role` can be a simple enum column on `User` rather than a full join table, since V1 has only two roles and no need for dynamic role assignment beyond that (Section 4.2 already recommends keeping this extensible in code, which an enum with a documented migration path still allows). `NotificationLog` is recommended (not strictly required) purely for observability/debugging of email failures (Section 14.3) - a lightweight table, not a queue.

22.2 ER Diagram

```
erDiagram
    USER ||--o{ REGISTRATION : makes
    USER {
        uuid id PK
        string legacy_user_id
        string name
        string email UK
        string phone
        date date_of_birth
        int age_category_id FK
        string wilaya
        string password_hash
        string role
        timestamp created_at
        timestamp updated_at
    }
    AGE_CATEGORY ||--o{ USER : classifies
    AGE_CATEGORY {
        int id PK
        string label
        int min_age
        int max_age
    }
    TRAINING_TYPE ||--o{ SESSION : "is scheduled as"
    TRAINING_TYPE {
        uuid id PK
        string training_code UK
        string name
        text description
    }
    SESSION ||--o{ SESSION_TARGET_CATEGORY : targets
    SESSION {
        uuid id PK
        uuid training_id FK
        string title
        text description
        date date
        time start_time
        int duration_minutes
        int max_capacity
        int registered_count
        string zoom_url
        string instructor_name
        boolean is_cancelled
```

```
    timestamp cancelled_at
    timestamp created_at
    timestamp updated_at
}
AGE_CATEGORY ||--o{ SESSION_TARGET_CATEGORY : "targeted by"
SESSION_TARGET_CATEGORY {
    uuid session_id FK
    int age_category_id FK
}
SESSION ||--o{ REGISTRATION : receives
REGISTRATION {
    uuid id PK
    uuid user_id FK
    uuid session_id FK
    timestamp registered_at
}
REGISTRATION ||--o| ATTENDANCE : "results in"
ATTENDANCE {
    uuid registration_id PK, FK
    timestamp attended_at
    timestamp validated_at
    uuid validated_by FK
}
ATTENDANCE ||--o| CERTIFICATE : enables
CERTIFICATE {
    uuid id PK
    uuid registration_id FK
    timestamp issued_at
    uuid validated_by FK
}
USER ||--o{ CERTIFICATE : "validates (as session manager)"
```

22.3 Key Constraints and Indexes

Table	Constraint / Index
`user`	`UNIQUE(email)`; index on `legacy_user_id` for migration lookups; index on `age_category_id`
`session`	index on `(training_id)`; index on `(date, start_time)` for list ordering; `CHECK (registered_count <= max_capacity)`
`session_target_category`	composite PK `(session_id, age_category_id)`
`registration`	`UNIQUE(user_id, session_id)`; index on `session_id` (for capacity/count queries) and on `user_id` (for “My Sessions”)
`attendance`	PK = `registration_id` (enforces one-to-zero-or-one relationship, satisfying idempotency, Section 12.2)
`certificate`	`UNIQUE(registration_id)` (one certificate per attended registration)
`training_type`	`UNIQUE(training_code)`

---

23 API Design (Representative Endpoints)

23.1 Authentication

Method & Path	Purpose	Auth	Notes
`POST /auth/register`	New participant registration	None	Validates uniqueness of email; derives age category from DOB
`POST /auth/login`	Login	None	Returns access + refresh token
`POST /auth/refresh`	Refresh access token	Refresh token	Rotates refresh token
`POST /auth/logout`	Invalidate refresh token	Access token	
`GET /auth/me`	Current user profile	Access token	
`POST /auth/activate`	Activate migrated account / set initial password	Activation token (in body)	See Section 2.1
`POST /auth/password-reset/request`	Request reset email	None	Generic response regardless of email existing (Section 19)
`POST /auth/password-reset/confirm`	Set new password	Reset token (in body)	Invalidates other sessions

23.2 Profile

Method & Path	Purpose	Auth
`GET /users/me`	Get profile	Participant
`PATCH /users/me`	Update profile fields (Section 3.4)	Participant

23.3 Sessions

Method & Path	Purpose	Auth	Business rules
`GET /sessions`	List sessions eligible for current user's category, ordered per Section 7.2; excludes `zoom_url`	Participant	Filtered server-side by age category
`GET /sessions/{id}`	Session detail (no `zoom_url`)	Participant	404 if not eligible/not found
`POST /sessions`	Create session	Session Manager	Requires valid `training_id` (or creates new `TrainingType`), category, capacity, Zoom URL
`PATCH /sessions/{id}`	Edit session	Session Manager	See Section 38.20 for handling edits after registrations exist
`POST /sessions/{id}/cancel`	Cancel session	Session Manager	Triggers cancellation emails (async)

23.4 Registration

Method & Path	Purpose	Auth	Errors
POST /sessions/{id}/register	Register (the "Attend" action)	Participant	403 ineligible category, 409 already registered, 409 capacity full, 410 session completed/cancelled
DELETE /sessions/{id}/register	Unregister	Participant	403 if past the 1-hour cutoff (Section 10)
GET /users/me/sessions	"My Sessions" list with derived states (Section 7.3)	Participant	

23.5 Attendance / Join

Method & Path	Purpose	Auth	Notes
POST /sessions/{id}/join	Validates eligibility + registration + time window; records attendance idempotently; returns zoom_url	Participant	The **only** endpoint that ever returns zoom_url
GET /sessions/{id}/attendance-status	Whether current user has attended	Participant	

23.6 Certificates

Method & Path	Purpose	Auth
GET /certificates/{registration_id}	Validated data for client-side rendering (Section 13.3)	Participant (owner only)

23.7 Session Manager

Method & Path	Purpose	Auth
GET /sessions/{id}/stats	Registered/Attended/Absent counts (Section 16)	Session Manager
POST /sessions/{id}/attendance/validate	Bulk-validate attendance → creates Certificate rows for attended registrations	Session Manager

All endpoints return a consistent error envelope (Section 29) and use standard HTTP status codes (400, 401, 403, 404, 409, 410, 422, 500).

---

24 Business Rules

#	Rule
1	A user must have an account to register for a session.
2	Age category is always derived from date of birth; never manually set (Section 3.3).
3	A participant may register for any number of different sessions.
4	A participant cannot register twice for the same session (`UNIQUE(user_id, session_id)`).
5	Capacity (`max_capacity`) can never be exceeded, even under concurrent requests (Section 9.1).
6	Registration is possible any time before `start_time - 60min` , subject to capacity and eligibility.
7	Unregistration is possible only before `start_time - 60min` .
8	Zoom access (and therefore attendance eligibility) opens exactly at `start_time - 60min` and remains open through session completion, unless cancelled.
9	Attendance is recorded only via the `/join` endpoint, on first successful call.
10	Repeated `/join` calls do not create duplicate attendance or change `attended_at` .
11	Registration never implies attendance; the two are always tracked and displayed separately.
12	A registered participant who never joins is `Absent` once the session is `COMPLETED` .
13	Only participants with **validated** attendance receive a `Certificate` .
14	Certificates are tied to a specific `Registration` (hence a specific participant + session + training type).
15	Session status is computed from time (`OPEN` / `RUNNING` / `COMPLETED`), except `CANCELLED` , which is an explicit stored override (Section 15.1).
16	Cancelled sessions cannot be joined, registered for, or unregistered from.
17	Participants may freely attend multiple sessions of the same `TrainingType` ; prior attendance is informational only in V1 (Section 6.3).
18	Repeated attendance of the same `TrainingType` is identifiable via the `Session.training_id` FK and is surfaced as a UI indicator only.

24.1 Identified Ambiguities and Resolutions

- **Ambiguity:** “registration possible until one hour before start” vs. “capacity reached” - which check wins if both would otherwise pass but capacity is reached exactly at the boundary? **Resolution:** capacity is checked transactionally at insert time (Section 9.1) regardless of the time-based rule; both are independent gates that must both pass.
- **Ambiguity:** what happens to existing registrations if a session’s date/time is edited after people have registered? Not stated in the source requirements. **Resolution:** addressed as an open design decision, Section 38.20.
- **Ambiguity:** what happens if a participant’s date of birth edit changes their age category **after** they’ve registered for a session whose target category no longer matches them? **Resolution:** addressed in Section 38.19 - existing registration is **not** retroactively revoked; only future eligibility checks use the new category.

---

25 Use Cases (Representative Selection)

> *The following two are shown in full detail as a template; all sixteen use cases follow the same structure and should be completed as part of detailed design review before sprint planning (Section 35).*

25.1 UC-01: New Participant Registration

- **Actor:** Anonymous visitor
- **Preconditions:** None
- **Main flow:** Visitor submits name, email, phone, DOB, wilaya, password → system validates uniqueness of email → system derives age category from DOB → account created → confirmation (and optionally a welcome email).
- **Alternative flows:** Email already exists → system suggests login/password-reset instead of exposing whether the account exists via a distinct error message that doesn't leak which specific field is a duplicate beyond "email already in use" (a deliberate exception to the account-enumeration guidance in Section 19, since it applies specifically to the login/reset flow, not registration, where the trade-off favors clear UX).
- **Exceptions:** Invalid DOB format, weak password (validation errors, `422`).
- **Postconditions:** New `User` row exists with role `PARTICIPANT`.

25.2 UC-06: Registering for a Session

- **Actor:** Authenticated participant
- **Preconditions:** Logged in; session is `OPEN` or has remaining capacity; participant's category matches.
- **Main flow:** Participant views session detail → clicks "Attend" → backend runs the six checks (Section 9.1) inside a transaction → `Registration` created → `registered_count` incremented → UI updates to `Registered`.
- **Alternative flows:** Capacity reached at the moment of click → `409`, button shown as "Registration Full."
- **Exceptions:** Session cancelled/completed between page load and click → `410`.
- **Postconditions:** `Registration` row exists; session now visible under "My Sessions."

(Use cases UC-02 through UC-16 - existing participant migration/activation, login, profile modification, viewing sessions, unregistering, viewing session details, accessing Zoom, recording attendance, viewing My Sessions, downloading a certificate, and the four session-manager use cases - follow directly from the business rules in Section 23 and the endpoint definitions in Section 22, and should be written out in the same actor/precondition/flow/exception/postcondition structure during sprint planning once the design decisions in Section 38 are confirmed, to avoid documenting flows that would need to be rewritten after those decisions are made.)

---

26 User Journeys

```
flowchart LR
    A1[Register] --> A2[Complete profile] --> A3[Browse sessions] --> A4[Register for session] --> A5[Join at session time] --
> A6[Attendance recorded] --> A7[Certificate available]
```

New participant: Register → Profile → Session discovery → Registration → Session access → Attendance → Certificate.

Existing (migrated) participant: Receive activation email → Activate/set password → Login → Session discovery → Registration → Attendance → Certificate.

Session manager: Login → Create session (select/create Training Type) → Publish → Monitor registrations (`N/Capacity`) → Session begins (auto → RUNNING) → Monitor attendance in real time → Validate attendance after completion → Certificates become available to participants.

---

27 State Diagrams

27.1 Session Lifecycle

```
stateDiagram-v2
    [*] --> OPEN
    OPEN --> RUNNING: start_time reached
    RUNNING --> COMPLETED: start_time + duration reached
    OPEN --> CANCELLED: manager cancels
    RUNNING --> CANCELLED: manager cancels
    CANCELLED --> [*]
    COMPLETED --> [*]
    note right of OPEN
        FULL is a derived display
        condition (registered_count >= max_capacity),
        not a stored state
    end note
```

`OPEN`, `RUNNING`, `COMPLETED` are genuine, time-derived lifecycle states. `CANCELLED` is a stored, explicit override state. `FULL` is a derived UI-only condition layered on top of `OPEN` (Section 8.2).

27.2 Participant–Session Relationship

```
stateDiagram-v2
    [*] --> NoRegistration
    NoRegistration --> Registered: register()
    Registered --> NoRegistration: unregister() [before cutoff]
    Registered --> Attended: join() [first click, within window]
    Registered --> Absent: session completes, no join
    Attended --> CertificateAvailable: manager validates attendance
    Registered --> Cancelled: session cancelled
```

---

28 System Architecture

```
flowchart TD
  subgraph Client
    RC[React SPA]
  end
  subgraph Server["Organization-managed server"]
    NG[Nginx reverse proxy / TLS termination]
    API[FastAPI application - modular monolith]
    SCH[In-process scheduler: status/reminders]
    DB[(PostgreSQL)]
    SMTP[SMTP relay - existing org infra]
  end
  RC -->|HTTPS / JSON| NG --> API
  API -->|SQL, transactions| DB
  SCH -->|reads/writes| DB
  SCH -->|enqueue/send| SMTP
  API -->|send confirmation/cert emails| SMTP
```

- **Synchronous:** all participant/session-manager REST interactions (registration, join, profile, session CRUD).
- **Asynchronous:** email sending (fire-and-forget from the request path into a lightweight job queue/table, Section 28).
- **Scheduled:** reminder emails, and optionally a periodic sanity job (no scheduled job is required for session status itself, since that is computed on read - Section 8.3).

Handling scale (Section 17): a single PostgreSQL instance with the indexes in Section 21.3 comfortably serves 23,000+ users and dozens of sessions/month; the `SELECT ... FOR UPDATE` pattern (Section 9.1) handles registration bursts safely; the `/join` burst in the hour before a 300-person session is a read-mostly, low-write workload well within a single small server's capacity. If growth exceeds this, the modular monolith can be scaled horizontally (multiple Uvicorn workers/replicas behind Nginx) without an architecture rewrite, and the clear module boundaries (Section 18) leave room to extract a service later if ever justified - without doing so speculatively now.

---

29 Background Tasks and Automation

Task	Trigger	Recommended mechanism
Session reminder emails	Scheduled (e.g., every 15 min, check for sessions starting in 24h / 1h)	In-process scheduler (APScheduler) running inside the FastAPI process, or a simple cron-triggered script hitting an internal endpoint
Cancellation notifications	Event-driven (on `POST /sessions/{id}/cancel`)	Enqueued immediately from the request handler into a lightweight outbox table, processed by a small worker loop
Email sending (all types)	Outbox pattern: write an `email_outbox` row in the same transaction as the triggering action; a background loop polls and sends	Simple `asyncio` background task or cron script - no external queue broker required at this volume (dozens of sessions/month, thousands of emails at most per event)
Certificate rendering	On-demand at download time (client-side, from validated data) - no background job needed (Section 13.3)	N/A

Recommendation: No Celery/Redis. At 24 sessions/month and a user base in the tens of thousands, message volume is far below what justifies a broker-based queue. An **outbox table + a simple polling worker** (either a background `asyncio` task inside the same FastAPI process, or a small separate script run via systemd timer/cron) gives durability (nothing is lost if the process restarts mid-send) and retry capability with a fraction of the operational complexity, matching the "avoid a queue merely because it's popular" instruction.

---

30 Error Handling

All API errors use a consistent envelope, e.g.:

```
{
  "error": {
    "code": "SESSION_FULL",
    "message": "This session has reached its maximum capacity.",
    "details": {}
  }
}
```

Error condition	HTTP status	Error code
Invalid credentials	401	`INVALID_CREDENTIALS`
Duplicate email at registration	409	`EMAIL_ALREADY_EXISTS`
Invalid date of birth	422	`INVALID_DATE_OF_BIRTH`
Session not found	404	`SESSION_NOT_FOUND`
Registration closed (past cutoff / not yet open)	403	`REGISTRATION_CLOSED`
Capacity reached	409	`SESSION_FULL`
Duplicate registration	409	`ALREADY_REGISTERED`
Unauthorized Zoom access	403	`ZOOM_ACCESS_DENIED`
Session cancelled	410	`SESSION_CANCELLED`
Certificate not available	404	`CERTIFICATE_NOT_AVAILABLE`
Email delivery failure (internal, logged, not surfaced to the triggering request)	• (async, not part of the request/response cycle)	`EMAIL_SEND_FAILED` (internal log only)

The React frontend maps `error.code` to user-facing messages via a single shared lookup table, so backend and frontend error vocabularies stay in sync without string-matching on `message`.

---

31 Testing Strategy

Layer	Coverage
Backend unit	Business rule functions in isolation (age-category derivation, status computation, cutoff-time calculations).
Backend integration	Endpoint tests against a real (test) PostgreSQL database, covering the full registration/attendance/certificate flow.
Backend concurrency	Dedicated tests that fire many simultaneous registration requests at a near-full session and assert `registered_count` never exceeds `max_capacity` (Section 9.1, Section 36).
Frontend component	Individual components (session card, registration button states, My Sessions list).
Frontend integration	Key flows using mocked API responses (registration, join, certificate download).
End-to-end	Registration → login → session registration → capacity-full behavior → Zoom access window → attendance recording → certificate availability, run against a staging environment.
Security	Authentication/authorization boundary tests (role checks on every session-manager endpoint), input validation fuzzing, and a checklist review ahead of the organization's own penetration test (this document does not replace that test).
Concurrency-specific	Explicit load test simulating simultaneous "Attend" clicks near capacity, and simultaneous "Join" clicks at session start, matching the acceptance criteria in Section 36.

32 Deployment

```
flowchart LR
  Internet -->|HTTPS 443| Nginx
  Nginx -->|reverse proxy| Uvicorn[Uvicorn/Gunicorn workers - FastAPI]
  Uvicorn --> PG[(PostgreSQL)]
  Uvicorn --> SMTP[Org SMTP relay]
```

- **Reverse proxy:** Nginx handles TLS termination, static asset serving for the built React app, and proxies `/api` to the FastAPI application.
- **Application server:** Uvicorn workers managed by systemd (or Gunicorn with Uvicorn workers for process management/restarts).
- **Database:** PostgreSQL on the same or a dedicated organization-managed host, with scheduled `pg_dump` backups (and WAL archiving if higher recovery granularity is desired) plus periodic restore-testing.
- **Environment variables/secrets:** `.env`-style configuration loaded via Pydantic settings, with production secrets injected via the server's environment (or an organization secret manager, if available) rather than committed files.
- **Migrations:** Alembic migrations run as an explicit deployment step, never auto-applied silently in production without review.
- **Containerization - Docker or not? Recommendation: use Docker**, even for a single organization-managed server, specifically because it (a) makes the deployment reproducible and reviewable ahead of the organization's security team's penetration test, (b) isolates the Python/Node build environment from whatever else may run on the shared server, and (c) simplifies the two developers' local-development parity with production. This is a light-weight, single-`docker-compose.yml` (app + Postgres, or app only if Postgres is centrally managed) setup, not a container-orchestration platform - no Kubernetes is justified at this scale.
- **Update strategy:** simple rolling restart (brief downtime acceptable given the batch, session-scheduled nature of usage) or, if zero-downtime is desired, two Uvicorn worker sets behind Nginx swapped during deploy - **TBD**, depending on the organization's tolerance for brief maintenance windows.

---

33 Data Migration

Step	Description
1	Obtain approved export/access from the legacy platform team - mechanism **TBD** (Section 2).
2	Map legacy fields to new schema: `legacy_user_id → user.legacy_user_id`, name/email/phone/age-category mapped directly; **date of birth is likely not present in the legacy system** (since it stores age category directly) - flagged as **TBD/open question for the legacy team**: request DOB if available, otherwise the new system must accept migrated users without a DOB initially and prompt them to supply it on first login (needed since the new system derives category from DOB going forward, per Section 3.3).

V1 does allow a `date_of_birth` to be nullable for migrated accounts until the user supplies it, with `age_category` populated directly from the legacy value in the interim. | | 3 | Validate data: required fields present, well-formed emails/phones. | | 4 | Identify duplicates (e.g., same email appearing more than once in the export). | | 5 | Validate email deliverability where feasible (format checks; full deliverability validation is out of scope for V1). | | 6 | Import users into a staging database first, not directly into production. | | 7 | Handle passwords per Section 2.1 (prefer activation-link flow). | | 8 | Verify row counts match the source export (reconciliation report). | | 9 | Run migration tests in a staging environment with a representative data sample. | | 10 | Perform the final migration during a scheduled maintenance window. | | 11 | Verify a sample of migrated accounts (login/activation flow, category correctness) before general announcement. |

This process is intentionally described independently of the legacy platform's underlying database technology, per the explicit instruction not to assume it.

---

34 V1 Scope Exclusions and Rationale

Excluded feature	Rationale
Quizzes, courses, video recording	Not part of the current manual workflow being automated; belongs to the future LMS direction (Section 34).
Chat	No manual equivalent exists in the current process to automate.
Payment	Training sessions are free/internal; no billing requirement stated.
Mobile application	React web app is responsive and sufficient for V1; native apps add build/release overhead disproportionate to the problem.
AI recommendation	No manual equivalent; premature without usage data.
Advanced analytics	Only the specific aggregate counts (registered/attended/absent) tied to eliminating manual attendance tracking are needed.
Social features	Not part of the training-session workflow.
Participant search/filtering/export	Not required to run a session; deferred until a genuine administrative need beyond "manage sessions" is identified.
Advanced attendance correction	The entire point of automatic, click-based attendance is to remove manual correction; introducing a correction UI in V1 would partially reintroduce the manual burden being eliminated.
Automated Zoom meeting creation via API	Session managers already create meetings manually today; automating this is a genuine future improvement but not required to eliminate the **participant-facing** manual steps this project targets first.
Complex certificate template management	A single, fixed, well-designed template satisfies the requirement; a management UI is speculative infrastructure for a need that hasn't been demonstrated.

35 Future V2 Direction (Non-Binding)

The long-term direction may evolve toward a Coursera-like platform (learning pathways/tracks, courses, recorded sessions, quizzes, progress tracking, richer analytics). V1's architecture avoids foreclosing this by:

- Keeping `TrainingType` as a first-class entity distinct from `Session` (Section 6), which is exactly the seed of a future "course" or "track" concept.
- Using a role model (Section 4.2) that can be extended with additional roles (e.g., `INSTRUCTOR`, `ADMIN`) without restructuring existing tables.
- Keeping the API versioned/namespaced (e.g., `/api/v1/...`) so V2 additions do not require breaking existing clients.
- Not hard-coding "one training = one Zoom session" anywhere beyond the V1 UI layer - the underlying `TrainingType` → `Session` relationship already supports multiple future session types (in-person, recorded, self-paced) if a `session_type` field is added later.

No further V2 design is undertaken here, per the explicit instruction to avoid designing it in detail.

---

36 Implementation Roadmap

Phase	Objectives	Major tasks	Dependencies	Deliverables	Acceptance criteria
1. Requirements validation	Resolve all Section 38 decisions and TBD items	Stakeholder review of this document; confirm age-category table, legacy password format, SMTP availability	This document	Signed-off SRS	All open questions have documented answers
2. Architecture & database	Finalize schema	Implement schema (Section 21), Alembic migrations, seed `AgeCategory` table	Phase 1	Migration scripts, ER diagram matching implementation	Schema reviewed against Section 21
3. Backend foundation	Project skeleton	FastAPI app structure, config management, logging, DB connection pooling	Phase 2	Running skeleton with `/health`	Skeleton deployable to staging
4. Authentication	Login/registration/activation/reset	JWT issuance, password hashing, token endpoints	Phase 3	Auth endpoints (Section 22.1)	Security checklist (Section 19) passes for auth flows
5. User/profile	Profile CRUD	`/users/me` endpoints, age-category derivation logic	Phase 4	Profile endpoints	Editing DOB recomputes category correctly
6. Session management	Session Manager CRUD	`TrainingType` / `Session` models and endpoints, category targeting	Phase 2	Session CRUD endpoints	Session Manager can create/edit/cancel a session
7. Registration	Registration/unregistration	Transactional capacity logic (Section 9.1), cutoff enforcement	Phase 6	Registration endpoints	Concurrency test (Section 36) passes
8. Attendance/Zoom access	Join flow	`/join` endpoint, idempotent attendance, time-window checks	Phase 7	Attendance endpoints	Attendance acceptance criteria (Section 36) pass
9. Certificate	Validation + data endpoint	`/certificates/{id}` endpoint, validation endpoint,	Phase 8	Certificate flow	Certificate acceptance crite-

		frontend rendering template			ria (Section 36) pass
10. Email automation	Reminders/cancellation/activation/reset emails	Outbox table + polling worker (Section 28)	Phases 4, 6, 8	Working email pipeline	Emails sent asynchronously without blocking core actions
11. React frontend	All participant + manager screens	Auth screens, dashboard, session pages, registration UX, My Sessions, Session Manager UI	Phases 4–9 (incrementally)	Functional SPA	All use cases (Section 24) demonstrable end-to-end
12. Integration	Wire frontend to real backend	Replace mocks, error-envelope handling (Section 29)	Phase 11	Integrated staging build	E2E smoke tests pass
13. Testing	Full test suite	Unit, integration, E2E, concurrency (Section 30)	Phase 12	Test suite with CI run	Coverage of critical flows; concurrency tests pass
14. Security testing	Internal review ahead of pentest	Checklist review (Section 19), dependency audit	Phase 13	Security review notes	No critical/high findings outstanding
15. Deployment	Production environment setup	Docker/compose, Nginx, TLS, backups (Section 31)	Phase 14	Deployed staging → production environment	Staging mirrors production configuration
16. Migration	Execute legacy data import	Follow Section 32 process	Legacy team's export mechanism (TBD)	Migrated user base in production	Row-count reconciliation matches source
17. Production launch	Go-live	Final smoke test, announcement	Phase 15, 16	Live system	All Section 36 acceptance criteria verified in production

37 Acceptance Criteria (Representative)

- **Registration/capacity:** Given a session with capacity 100 and 99 registered participants, two concurrent registration requests result in exactly one success and one `SESSION_FULL` rejection - never two successes.
- **Unregistration cutoff:** Given a session starting at 14:00, an unregistration request at 13:00:01 is rejected; a request at 12:59:59 succeeds.
- **Zoom access:** Given a registered participant and a session at 14:00, `/join` returns `ZOOM_ACCESS_DENIED` before 13:00 and returns the Zoom URL with a recorded attendance event from 13:00 onward, provided the session is not cancelled.
- **Attendance idempotency:** Given a registered participant who calls `/join` twice in succession, exactly one `Attendance` row exists, with `attended_at` equal to the time of the first call.
- **Absence:** Given a registered participant who never calls `/join` before the session's `COMPLETED` status is reached, "My Sessions" displays `Absent` for that session.
- **Certificate availability:** Given a participant with a **validated** `Attendance` record, `GET /certificates/{registration_id}` returns the certificate data; given an unvalidated attendance, it returns `CERTIFICATE_NOT_AVAILABLE`.
- **Cancellation:** Given a `CANCELLED` session, `/join` and `/register` both reject with `SESSION_CANCELLED` / `410`, and all previously registered participants have a corresponding cancellation email enqueued in the outbox.
- **Age-category derivation:** Given a date of birth corresponding to age 16, the computed category is `15–17`, and this value is read-only in the profile UI.

---

38 Requirements Traceability Matrix (Representative Sample)

ID	Category	Description	Priority	Actor	Use case	Component	Acceptance criteria ref
REQ-01	Functional	Participant self-registration with DOB-derived category	Must Have	Participant	UC-01	Auth module	Section 36, age-category
REQ-02	Functional	Session visibility filtered/ordered by category and status	Must Have	Participant	UC-05	Sessions module	Section 7
REQ-03	Functional	Concurrency-safe registration with capacity enforcement	Must Have	Participant	UC-06	Registration module	Section 36, registration/capacity
REQ-04	Functional	Unregistration disabled from T-60min	Must Have	Participant	UC-07	Registration module	Section 36, unregistration cutoff
REQ-05	Functional	Zoom URL served only via authenticated /join	Must Have	Participant	UC-09	Attendance module	Section 36, Zoom access
REQ-06	Functional	Idempotent attendance recording on first join	Must Have	Participant	UC-10	Attendance module	Section 36, attendance idempotency
REQ-07	Functional	Certificate available only after validated attendance	Must Have	Participant / Session Manager	UC-12, UC-16	Certificate module	Section 36, certificate availability
REQ-08	Functional	Session creation/edit/cancel by Session Manager	Must Have	Session Manager	UC-13–15	Sessions module	Section 36, cancellation
REQ-09	Functional	One-time migration	Must Have	System	UC-02	Migration process	Section 32

		of legacy accounts					
REQ-10	Technical	Modular monolith on FastAPI + PostgreSQL + React	Must Have	-	-	Architecture	Section 18, 27
REQ-11	Security	Password hashing, RBAC, HTTPS, rate limiting	Must Have	-	-	Auth/security	Section 19
REQ-12	Should Have	Asynchronous reminder/cancellation emails	Should Have	Participant	UC-06, UC-15	Notification module	Section 14, 28
REQ-13	Could Have	Certificate email delivery	Could Have	Participant	UC-12	Notification module	Section 14
REQ-14	Future	Automated Zoom meeting creation	Future	Session Manager	-	-	Section 34

(This matrix is a representative starting set; the full matrix should enumerate every rule in Section 23 and every endpoint in Section 22 individually before sprint planning.)

---

39 Key Design Decisions - Summary

#	Decision	Recommendation	Reference
1	FULL: stored or derived?	Derived from `registered_count` / `max_capacity`	8.2
2	Training Type vs. Session as separate entities?	Yes, separate entities with FK	6.2
3	Attendance: separate entity or fields on Registration?	Separate `Attendance` entity	12.3
4	Age category: stored or derived?	Derived from DOB, recomputed on DOB edit	3.2
5	Manual age-category override allowed?	No - read-only, DOB is the only editable input	3.3
6	Migrated password handling	Prefer secure activation-link flow over hash migration	2.1
7	Certificate generation: client vs. server side?	Hybrid: client-rendered from a server-validated data endpoint	13.3
8	Certificate generation timing	Immediately available once Session Manager validates attendance	13.1
9	Email architecture	Existing/organization SMTP + outbox-table pattern	14.2, 28
10	Background job system needed?	No broker; in-process scheduler + outbox polling worker suffices	28
11	Session status: dynamic or persisted?	Dynamic (computed on read) except `CANCELLED`, which is stored	8.3, 15.1
12	Cancellation effect on registrations	Registrations preserved (not deleted) with `CANCELLED` status; notification emails triggered	15.2
13	Repeated Training ID representation	`Session.training_id` FK to `TrainingType`; surfaced as a UI badge only	6.2, 6.3
14	Concurrent registration protection	`SELECT ... FOR UPDATE` row lock + `UNIQUE(user_id, session_id)` + capacity re-check in one transaction	9.1
15	Authentication mechanism	Short-lived JWT access token + rotating refresh token	18, 19
16	Zoom URL protection	Never in list/detail responses or emails; served only via authenticated, re-validated `/join` endpoint	11.2, 22.5
17	Email delivery failure handling	Logged and retried asynchronously; never blocks or rolls back the triggering business action	14.3
18	Certificate generation failure handling	Data endpoint failure is retried client-side on demand; no state is lost since generation is on-demand, not a one-time batch job	13.3
19	Category change after registration	Existing registration is preserved; new eligibility rules apply only to future registrations	23.1
20	Session date/time/duration edited after registrations exist	**TBD** - recommend: allow edits, but require the Session Manager to explicitly confirm,	

		and trigger an automatic notification email to all currently registered participants informing them of the change; the registration itself is preserved (not auto-cancelled)	

40 Appendix A - Key Assumptions

- The legacy platform's data export mechanism, password hashing algorithm, and whether it stores date of birth are all currently unknown and must be confirmed with that team before Phase 16 (migration) begins.
- The organization's SMTP/email infrastructure availability for this new application is unconfirmed.
- The age-category table as given (stopping at 46–60) may be incomplete relative to the stated 7–77 audience and should be confirmed.
- No requirement for real-time (WebSocket) updates was identified; standard request/response polling (e.g., refreshing `registered_count` on page load/periodic refresh) is assumed sufficient for V1.
- "Deployment on an organization-managed server" is assumed to mean a single server (or a small number of servers) under Algérie Télécom's operational control, not a third-party cloud platform; this shapes the Docker/Nginx/systemd recommendations in Section 31.

41 Appendix B - Open Questions / TBD Items (Consolidated)

1. Should a “61–77” age category exist? (Section 1.1)
2. What is the legacy platform’s password hashing algorithm, if any? (Section 2.1)
3. What migration export mechanism can the legacy team provide? (Section 2, Section 32)
4. Does the legacy platform store date of birth, or only age category? (Section 32, step 2)
5. Does the organization provide SMTP infrastructure the new application can use? (Section 14.2)
6. Do session managers need visibility into participant PII (phone, etc.) beyond aggregate counts? (Section 19)
7. What are the applicable Law 18-07 requirements regarding consent, retention, and data subject rights? (Section 19.1)
8. What is the organization’s tolerance for brief downtime during deploys (affects rolling-update strategy)? (Section 31)
9. How should a session edit (date/time/duration) after registrations exist be handled procedurally - auto-notify only, or require re-confirmation from participants? (Section 38, decision 20)

42 Appendix C - Recommended Technology Stack (Summary)

FastAPI · SQLAlchemy 2.x + Alembic · PostgreSQL · Pydantic v2 · JWT (access + refresh) · React · Nginx · Docker (single/compose deployment, not orchestration) · pytest / React Testing Library / Playwright · in-process scheduler + outbox-table email pipeline (no Celery/Redis).

43 Appendix D - V1 / Future V2 Scope Recap

V1 automates: registration, eligibility, capacity-safe session registration/unregistration, in-platform gated Zoom access, click-based attendance, validated certificate issuance, reminder/cancellation emails, and a minimal session-manager dashboard - nothing more (full exclusion list in Section 33).

V2 direction (non-binding): evolution toward a broader learning platform (tracks, courses, recorded content, quizzes, richer analytics), enabled but not built by V1's `TrainingType` / `Session` separation and versioned API (Section 34).